

Verification Test Plan

This plan verifies the behavior of the components and interactions defined in the SDS (Sensor, Alert, ControlSystem, DataManager, ReportGenerator, RangerInterface) and the end-to-end workflows shown in your architecture and UML.

1) Unit Tests (granularity: single class/function)

UT-1 — `Alert.classify(level: string)`

Targeted feature: Sets/updates `Alert.classification`.

Vectors:

1. Create Alert with default `classification=""`; call `classify("definite")` → expect "definite".
2. Reclassify to "suspected" → expect "suspected".
3. Invalid input "maybe" → expect exception or rejected update (per spec decision); `classification` unchanged.

Coverage: Happy path and input validation on a core data object.

UT-2 — `ControlSystem` alarm transitions

Targeted features: `receiveAlert`, `triggerAlarm`, `stopAlarm`, `alarmState`.

Vectors:

1. Start with `alarmState=false`. Call `receiveAlert(Alert{soundType="mountain_lion"})` → expect `triggerAlarm()` called and `alarmState=true`.
2. Call `stopAlarm()` → expect `alarmState=false`.
3. Non-lion alert: `soundType="wind"` → expect no alarm (`alarmState` unchanged).

Coverage: Guard logic for alarm gating and deterministic state transitions.

UT-3 — `DataManager.storeAlert` / `retrieveAlerts` / `archiveSummary`

Targeted feature: Persistence + simple filtering.

Vectors:

1. Store three alerts; `retrieveAlerts({soundType:"mountain_lion"})` → returns only lion alerts.
2. Archive path: call `archiveSummary(Alert{timestamp=T0-31d})` then `retrieveAlerts({})` → archived item is absent from active store (or marked summarized), per design.

Coverage: CRUD semantics and retention hooks.

UT-4 — `ReportGenerator.generateByDate` / `generateByLocation` / `generateMountainLionReport`

Targeted feature: Deterministic, filter-based report content.

Vectors:

1. With 5 alerts across two days, `generateByDate(day=T0)` → only that day's alerts, correct counts.
2. `generateByLocation(loc="32.7157, -117.1611")` → set filtered to location.
3. `generateMountainLionReport()` → contains only alerts where `soundType="mountain_lion"` (and/or classification indicates lion), includes summary fields (count, time range).

Coverage: Filtering correctness and output schema.

2) Integration Tests (granularity: interacting modules)

IT-1 — `Sensor` → `ControlSystem` → `DataManager` (ingest pipeline)

Targeted feature: End-to-end ingest and save.

Setup: Sensor stub emits `Alert{soundType="mountain_lion", strength=0.8, timestamp=T0}`.

Steps & Expected:

1. `Sensor.sendAlert()` delivers to `ControlSystem.receiveAlert()` → `alarmState=true` (gated by lion rule).
2. `ControlSystem.saveAlert()` called → `DataManager.storeAlert()` persists record.
3. `DataManager.retrieveAlerts({soundType:"mountain_lion"})` returns the exact alert (id, sensorID, fields round-tripped).
Failures covered: dropped message, wrong field mapping, missed save.

IT-2 — Ranger classification loop

Targeted feature: `RangerInterface` ↔ `ControlSystem` update path.

Steps:

1. `RangerInterface` loads list via `viewAlerts()` → gets unclassified alert.
2. Ranger classifies via `classifyAlert(level="definite", rangerID=101)` → `Alert.classification` updated; provenance includes `rangerID`.
3. Persisted state reflects classification on subsequent `retrieveAlerts({classification:"definite"})`.
Failures covered: race on concurrent classifications, missing ranger metadata.

IT-3 — Report generation over persisted data

Targeted feature: `DataManager` ↔ `ReportGenerator`.

Steps:

1. Insert alerts across multiple dates/locations.
2. Call `generateByDate`, `generateByLocation`, and `generateByRanger(101)`; verify counts, included alert IDs, and headers/summary blocks.
Failures covered: wrong filter predicates, timezone mistakes, empty sections.

3) System Tests (granularity: whole system / black-box)

ST-1 — “Confirmed lion” end-to-end

Goal: A real lion detection flows through alarms, storage, classification, and reporting.

Procedure:

1. Sensor stub emits lion-like alert (high strength, valid location).
2. ControlSystem raises alarm; ranger acknowledges via UI (stop alarm).
3. Ranger classifies "definite".
4. Generate *Mountain Lion Report* and *By Date* report for today.

Pass criteria:

- Alarm was raised once; then cleared by stopAlarm.
- Alert persisted with all fields and classification="definite".
- Reports include this alert in correct sections and counts.

ST-2 — “Benign noise” suppression

Goal: Non-lion signals do not alarm and are handled gracefully.

Procedure:

1. Emit alerts with soundType="wind" and strength varying.
2. Observe ControlSystem.

Pass criteria:

- alarmState never becomes true.
- Alerts may be stored (if design calls for storing all alerts) but lion-only reports exclude them.

ST-3 — Retention & summarization policy

Goal: 30-day active storage + yearly summary.

Procedure:

1. Seed DB with alerts at *T0-29d*, *T0-31d*, *T0-11mo*.
2. Run retention job (summarizeOldAlerts / archiveSummary).

3. Query active alerts and yearly summary artifacts.

Pass criteria:

- *T0-31d* alert is archived/summarized; not returned in active queries.
- *T0-29d* remains active.
- Summary dataset includes counts/time-bucket fields as specified.

ST-4 — Fault tolerance scenarios

Goal: Graceful handling of common failures.

Scenarios:

1. **Sensor offline** during event → ControlSystem logs and continues; no crash.
2. **DB temporarily unavailable** on saveAlert → retry w/ backoff; if still failing, alert queued; on recovery, persistence completes exactly once (idempotent write).
3. **Malformed alert** (missing location) → input rejected and error surfaced; not persisted.
Pass criteria: Each failure is contained, reported, and system returns to stable state; no data corruption or duplicate alerts.